Bipedal Robotic Locomotion with Reinforcement Learning

Drew Burcher, Dan Torri, Nathan Wyatt

Problem Statement and Project Goal

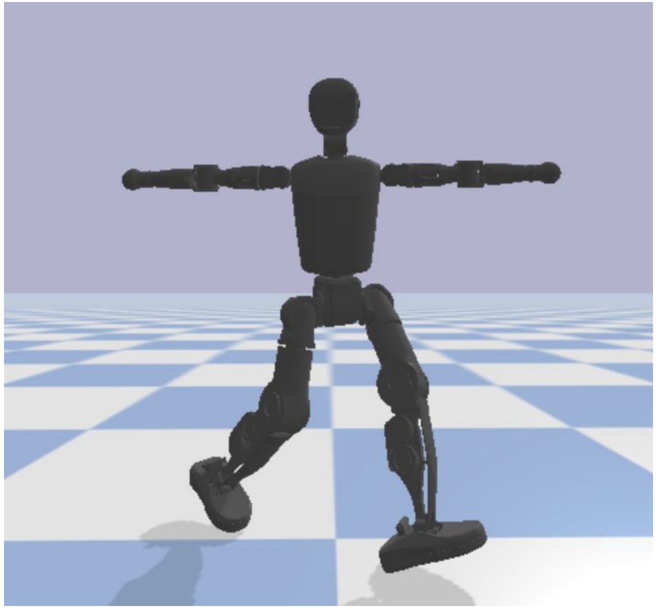
Goal: Train a bipedal robot to walk

Rewards: Speed and efficiency

Robot: 23 Joints total (13 actuated with arms locked)

Comparing two Reinforcement Learning Methods:

- SAC - Soft Actor Critic
- PPO - Proximal Policy Optimization



Simulation and Reward Setup

Simulation Setup:

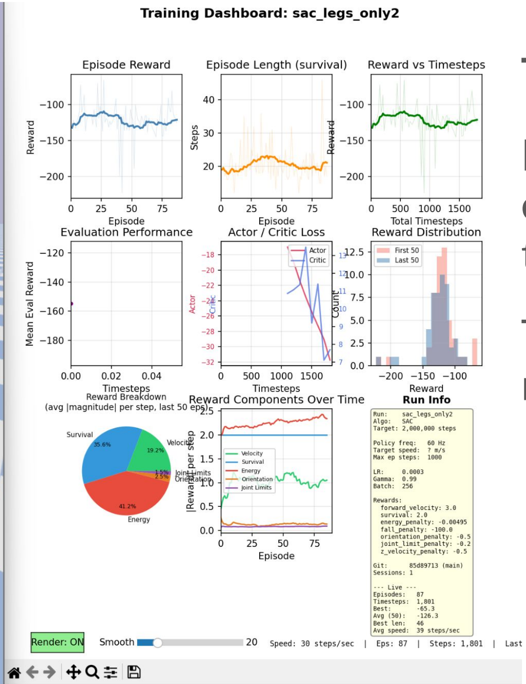
Pybullet wrapped in Gymnasium environment

Robot spawns in standing position

Robot resets if it falls

Primary Rewards:

- Forward Velocity
- Energy Efficiency

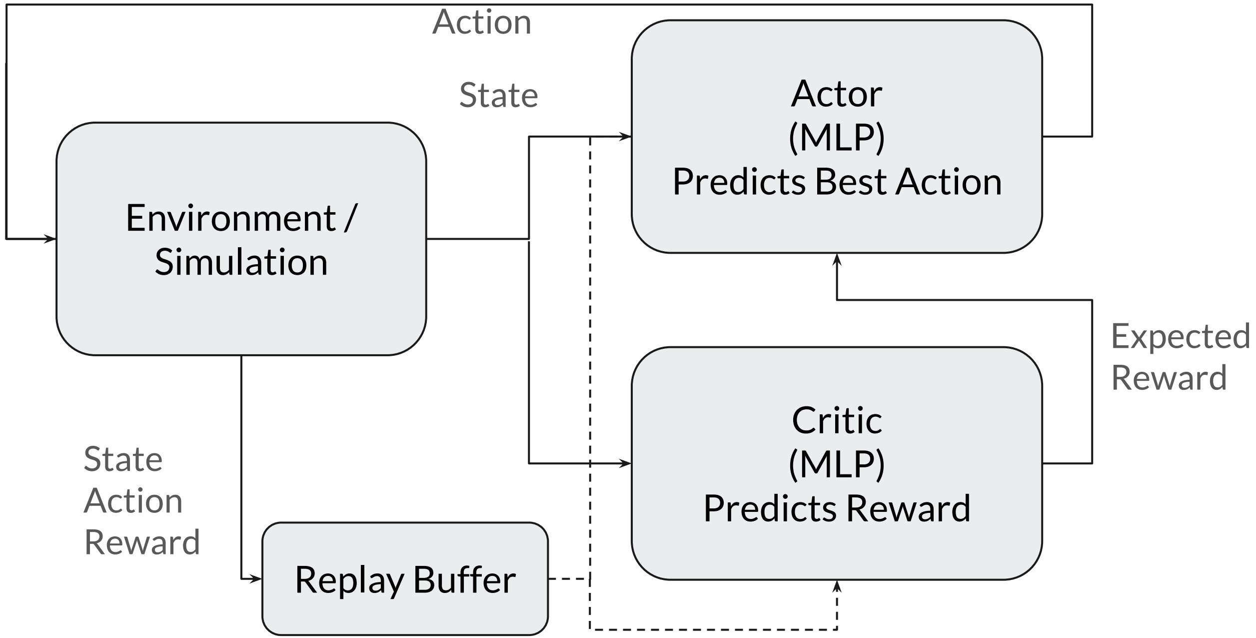


Training Setup:

Models are evaluated every 5k timesteps

Trained for ~3 million steps

Reinforcement Learning Methods



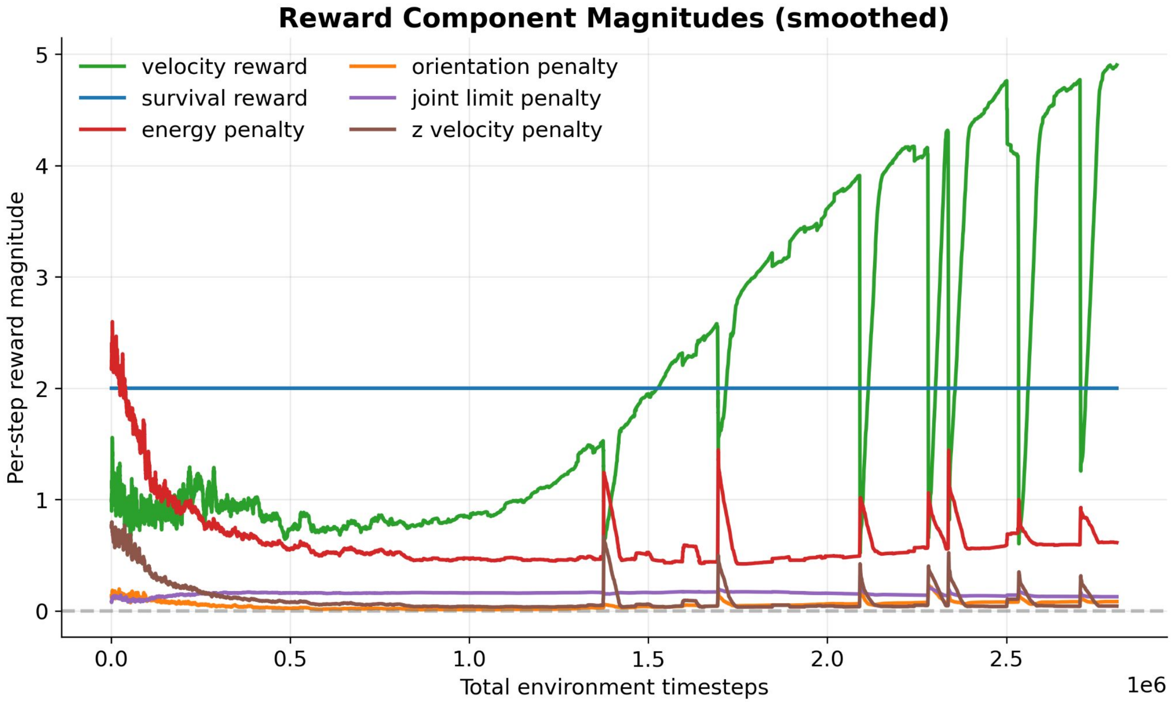
PPO:

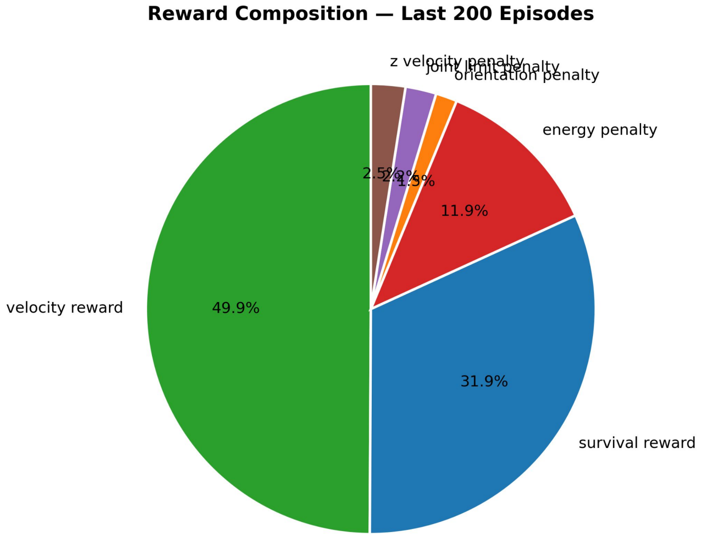
- On-policy
- Learns from recent rollouts
- More stable, but at cost of sample efficiency

SAC:

- Off-policy
- Learns from replay buffer
- More sample efficient, but more complex

Reward Breakdown (SAC)





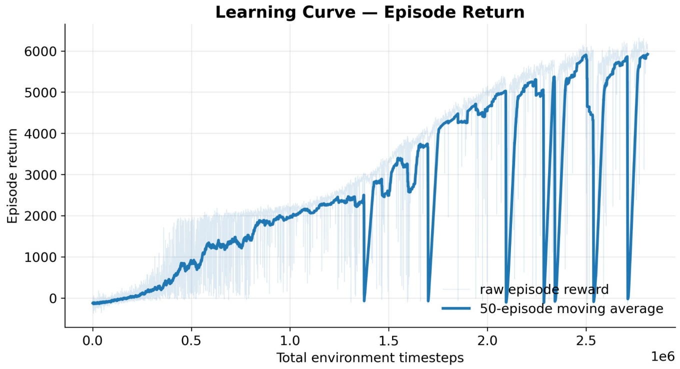
SAC Results

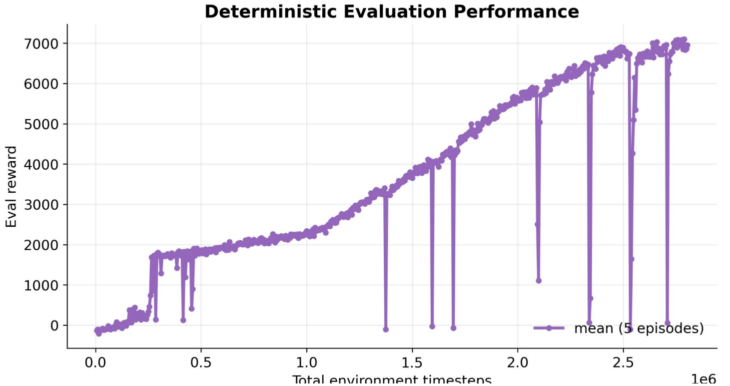
Results:

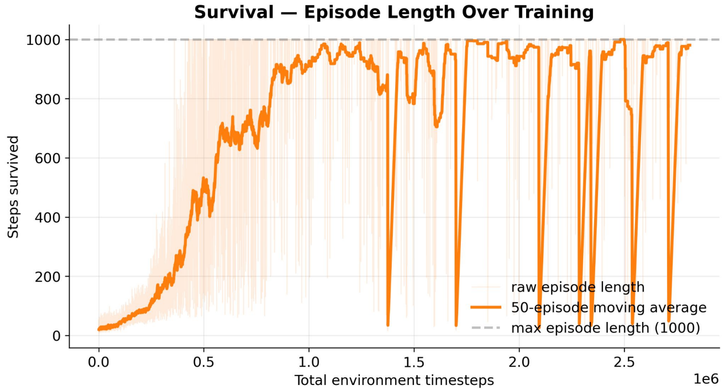
Performance plateaued after around 2.8 Million steps

Sharp increase in performance as robot learns to stand

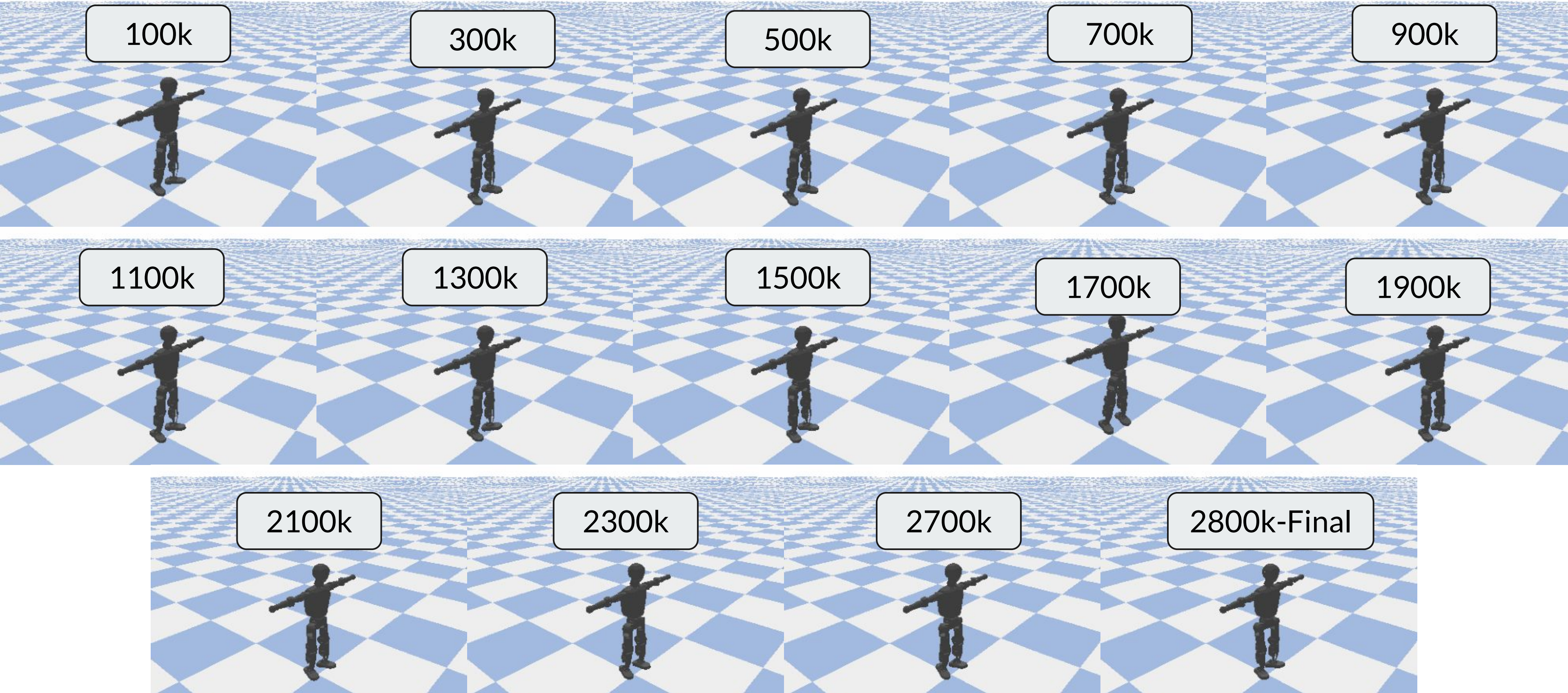
Occasionally the policy forgets how to balance



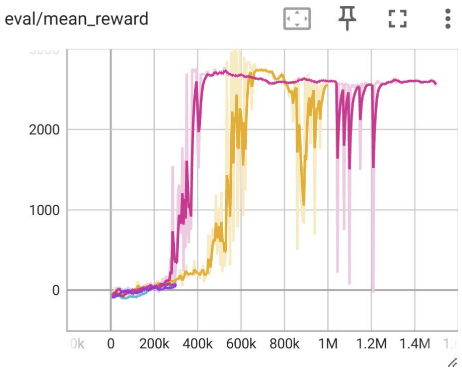




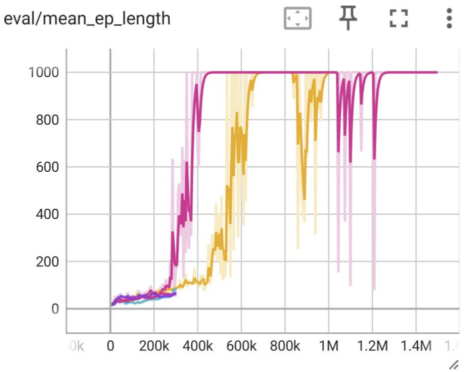
Training Progression: SAC



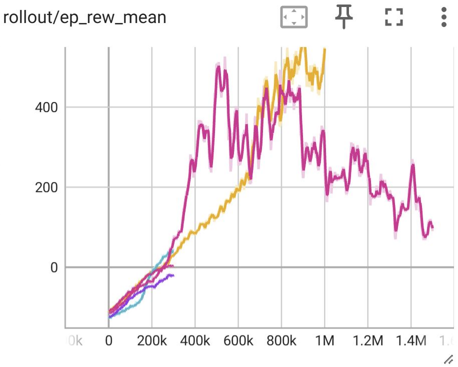
PPO Results



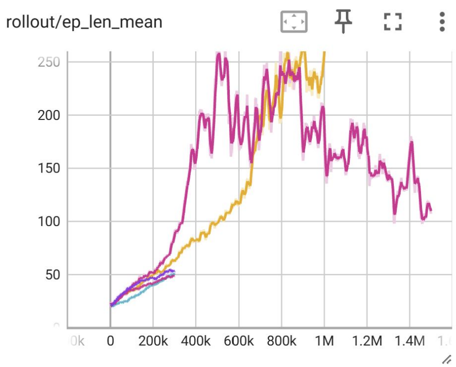
Run ↑	Smoothed	Value
ppo_t1_overnight_1m\tb\PPO_1	2,552.4498	2,551.75
ppo_t1_phase1\tb\PPO_2	83.0986	77.9296
ppo_t1_phase1_gui\tb\PPO_1	62.6347	47.8945
ppo_t1_targetspeed_v6\tb\PPO_1	2,560.3559	2,536.96
ppo_t1_targetspeed_v8\tb\PPO_1	70.9128	123.061



Run ↑	Smoothed	Value
ppo_t1_overnight_1m\tb\PPO_1	996.4238	1,000
ppo_t1_phase1\tb\PPO_2	77.9729	67
ppo_t1_phase1_gui\tb\PPO_1	57.9112	51
ppo_t1_targetspeed_v6\tb\PPO_1	1,000	1,000
ppo_t1_targetspeed_v8\tb\PPO_1	68.9813	81



Run ↑	Smoothed	Value
ppo_t1_overnight_1m\tb\PPO_1	546.2223	571.353
ppo_t1_phase1\tb\PPO_2	41.8723	41.9352
ppo_t1_phase1_gui\tb\PPO_1	3.6364	4.4354
ppo_t1_targetspeed_v6\tb\PPO_1	97.3916	90.231
ppo_t1_targetspeed_v8\tb\PPO_1	-20.5725	-19.2766



Run ↑	Smoothed	Value
ppo_t1_overnight_1m\tb\PPO_1	260.0696	269.54
ppo_t1_phase1\tb\PPO_2	51.231	52.18
ppo_t1_phase1_gui\tb\PPO_1	48.743	48.75
ppo_t1_targetspeed_v6\tb\PPO_1	109.7627	107.32
ppo_t1_targetspeed_v8\tb\PPO_1	53.3701	53.89

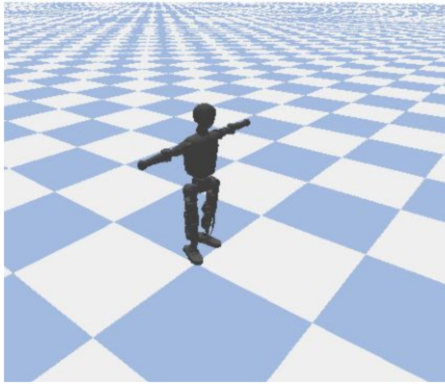
Training Progression: PPO

Early One Million time-step run:

- Too high reward for survival
- Too high reward for stability
- Too low reward for orientation
- No loss for lateral motion

Version 6 reward tuning Policy, 300k time-step run:

- Too high reward for survival
- Too high reward for stability
- Too low reward for forward velocity
- Too high loss for lateral motion



PPO vs. SAC Comparison

Training Method	PPO	SAC
Max Reward	~3000	~7100
Training Speed	Fast (~300 steps/s)	Slow (~50 steps/s)
On/Off Policy	On Policy (sample inefficient)	Off Policy (sample efficient)

Conclusions

- SAC outperformed PPO
- Rewards dramatically shaped learning outcome
- Robot learned how to move in unexpected ways

Questions?